

DEĞİŞKENLER

Görsel Programlama - Hafta 4

Giriş: Bilgiyi Saklamak

Bilişim Güvenliği Teknolojisi dersi kapsamında Görsel Programlama konularına odaklanmaktayız.

Bu hafta, programlamanın temel yapı taşı olan değişkenler ele alınacaktır. Konu başlığımız: Bilgiyi Saklamak: Değişkenler ve string Veri Tipi.



Ders Bağlamı ve Haftalık Konu

Bu içerik, Görsel Programlama dersinin 4. haftasına (HAFTA4) tekabül etmektedir.

Temel amacımız, verilerin programlama ortamında nasıl depolandığını ve işlendiğini anlamaktır.



Pedagojik Hedefler I

Bu bölümün sonunda öğrenciler, programlamanın en temel yapı taşı olan 'değişken' kavramını tam olarak kavrayacaklardır. Soyut tanımlar yerine güçlü analogilerle bu kavramın içselleştirilmesi hedeflenmektedir.



Pedagojik Hedefler II

Öğrenciler, incelenecek ilk veri tipi olan string veri tipini öğreneceklerdir. Metinsel verilerin programlama dilinde nasıl temsil edildiği anlaşılabacaktır.



Pedagojik Hedefler III

Öğrenciler, pratik düzeyde değişken tanımlama ve değişkenlere değer atama işlemlerini gerçekleştirebileceklerdir.

Metin birleştirme (String Concatenation) işlemlerini programatik olarak yapabilme becerisi kazanılacaktır.



Değişken (Variable) Nedir?

Değişken, bilgisayar programlarında geçici veya kalıcı veriyi saklamak için kullanılan isimlendirilmiş bir depolama alanıdır. Programlama dillerindeki verilerin dinamik olarak işlenmesini sağlayan kritik bir yapıdır.



Kritik Yaklaşım

Değişken kavramını öğrenirken soyut tanımlardan kaçınmak ve güçlü, somut analogiler kullanmak pedagojik olarak büyük önem taşır.



Analoji 1: Etiketli Kutular I

Değişkenler, üzerlerine etiket yapıştırılmış kutular gibi düşünülebilir.
Bu benzetmede 'kutu', bilgisayarın ana hafızasını (RAM) temsil etmektedir.



Analoji 1: Etiketli Kutular II

Kutunun üzerindeki 'etiket', değişkenin adını (örneğin isim) temsil eder. 'Kutunun içindeki' kısım ise değişkenin o an sakladığı değeri ifade eder (örneğin 'Ahmet').



Analoji 1: Değere Erişim

Bir değeri kullanmak istediğimizde, programlama dilinde sadece etiketin adını kullanırız.
Program, bu adı referans alarak hafızadaki kutunun içindeki değeri otomatik olarak getirir.

Analoji 1: Değerin Değiştirilmesi

Değişkenin temel özelliği, kutunun içindeki verinin yani değer değiştirilebilir olmasıdır.

Ancak etiketin adı (değişkenin ismi) genellikle sabit kalır ve programın ömrü boyunca bu isimle çağrılır.

Analoji 2: İleri Kavram

Daha ileri bir programlama anlayışı için farklı bir analogi gereklidir. Değişkenler, veriyi doğrudan 'içermez', sadece o veriye bir 'işaret' veya 'referans' görevi görürler.



Analoji 2: Referans Mekanizması

Bu analogide deęişkenler, bir değere takılmış isim etiketleri gibidir. Deęişkenler, o verinin bellekte nerede saklandığını gösteren işaretçilerdir.

Analoji 2: Çoklu Referans

Birden fazla farklı etiket (farklı değişken isimleri), aynı bellek adresindeki değere işaret edebilir.

Bu durum, özellikle referans tipleri (Reference Types) incelenirken büyük fayda sağlayacaktır.



Veri Tipi (Data Type) Kavramı

Veri tipi, 'etiketli kutu' analojisi üzerinden açıklanmalıdır.
Veri tipi, bir değişkenin yani kutunun içine ne tür bir bilgi koyabileceğimizi belirleyen kuraldır.



Veri Tiplerinin Sınıflandırılması

Bazı kutular sadece metinleri (string) saklamak için tasarlanmıştır. Bazı kutular tam sayıları (int), bazıları ise ondalıklı sayıları (double) saklamak için kullanılır.



Veri Tipinin Önemi

Veri tipi, bilgisayarın o değişkenle ne tür işlemler yapabileceğini bilmesini sağlar.

Örneğin, sayılarla matematiksel işlemler (toplama, çıkarma) yapılırken, metinlerle sadece birleştirme işlemleri gerçekleştirilir.

string Veri Tipi Detaylı İncelenmesi

string, metinsel karakter dizilerini ve verileri saklamak için özel olarak kullanılan bir veri tipidir.

Bu tip, isimler, adresler, açıklamalar veya kodlar gibi her türlü alfanümerik bilgiyi tutabilir.



string Değer Atama Sentaksı

string tipindeki bir değişkenin değeri tanımlanırken veya atanırken, metinsel ifade her zaman çift tırnak işareti (") içinde yazılmalıdır.

string Örnek Tanımlama

Örnek olarak bir şehir bilgisini saklamak için şu sentaks kullanılır:

```
string sehir = "Ankara";
```

Çift tırnaklar, derleyiciye içerideki ifadenin bir metin (string) olduğunu bildirir.

Değişken Tanımlama Sentaksı

C# programlama dilinde bir değişkenin tanımlanması şu genel formatı izler:

veriTipi degiskenAdi;

Değişken Tanımlama Örneği

string veri tipinde bir müşteri adı değişkeni tanımlama:
string musterAdi;
Bu adım, bellekte 'musterAdi' etiketiyle tanımlanmış bir string kutusu ayırır.

Değer Atama Operatörü

Tanımlanmış bir değişkene değer yükleme işlemi eşittir (=) operatörü ile gerçekleştirilir.

Bu operatör, sağdaki değeri alıp soldaki değişkene yerleştirir veya atar.

Değer Atama Örneği

Daha önce tanımlanan değişkene değer atama örneği:
`musteriAdi = 'Veli Yılmaz';`
Bu işlem, bellekteki `musteriAdi` kutusunun içine 'Veli Yılmaz' metnini koyar.

Kısa Yol Sentaksı

Programlamada en yaygın kullanılan yöntem, değişkeni tanımlama ve ilk değer atama işlemini tek bir satırda birleştirmektir.

```
string musteriAdi = 'Veli Yılmaz';
```

Metin Birleştirme (String Concatenation)

Artı (+) operatörü, matematiksel işlemlerin yanı sıra, string veri tipleriyle kullanıldığında farklı bir fonksiyona sahiptir. Bu fonksiyon, iki veya daha fazla metni yan yana ekleyerek tek bir metin oluşturmaktır.



Birleştirme Örneği I

```
string ad = 'Ali';
```

```
string soyad = 'Can';
```

Bu iki değişken, tamAd değişkenini oluşturmak için birleştirilecektir.

Birleştirme Örneği II

```
string tamAd = ad + ' ' + soyad;
```

Bu kod satırı çalıştırıldığında, tamAd değişkeninin değeri 'Ali Can' olacaktır. Metinler arasına boşluk eklemek için fazladan bir string (' ') eklenmelidir.

Kullanıcı Arayüzünde string Kullanımı

Görsel programlamada, kullanıcıya sonuç göstermek için Label (lbl) nesneleri kullanılır.

```
lblSonuc.Text = 'Hoş geldiniz, ' + tamAd;
```

Bu, statik bir metin ile dinamik olarak oluşturulmuş bir değişkenin birleştirilmesidir.

Uygulama Örneği Başlangıcı

Öğrenilen string ve değişken kavramlarını pratikleştirmek için bir 'Kişisel Tanıtım Kartı Formu' tasarlanacaktır.



Form Tasarımı Bileşenleri I

Forma kullanıcıdan veri almak için şu etiketler (Label) ve karşılıklarına metin kutuları (TextBox) eklenir:

'Ad:', 'Soyad:', 'Meslek:'.

TextBox nesnelerinin isimleri: txtAd, txtSoyad, txtMeslek.

Form Tasarımı Bileşenleri II

İşlemi başlatmak için bir Button (düğme) nesnesi eklenir.
Bu düğmeye 'Kart Oluştur' ismi verilir (btnOlustur).

Form Tasarımı Bileşenleri III

Oluşturulan metnin gösterileceği bir alan gereklidir.
Bu amaçla geniş, çok satırlı bir Label (IbIKart) veya Multiline özelliği 'true' yapılmış bir TextBox kullanılabilir.



Sonuç Ekranı Özellikleri

Eğer sonuç göstermek için TextBox kullanılıyorsa, kullanıcının bu metni değiştirmesini engellemek için ReadOnly özelliği 'true' yapılmalıdır.

Buton Olay Yönetimi

Tüm programatik mantık, 'Kart Oluştur' düğmesinin (btnOlustur) Click olayına yazılacak kodlar aracılığıyla çalıştırılacaktır.



Uygulama Adım 1

İlk olarak, TextBox'larda bulunan kullanıcı girişleri okunmalı ve string tipindeki değişkenlere atanmalıdır.

Bu, değişkenlerin rolünün somut bir uygulamasıdır.



Kodlama Adım 1.1: Adı Alma

```
string kullanıcıAdi = txtAd.Text;
```

Bu kod, txtAd isimli metin kutusunun içeriğini (Text özelliğini) alıp 'kullanıcıAdi' değişkenine atar.



Kodlama Adım 1.2: Soyadı Alma

```
string kullanıcıSoyadi = txtSoyad.Text;  
txtSoyad alanındaki metin, 'kullanıcıSoyadi' isimli string değişkende saklanır.
```

Kodlama Adım 1.3: Mesleği Alma

```
string kullanıcıMeslek = txtMeslek.Text;
```

Benzer şekilde, 'kullanıcıMeslek' değişkeni txtMeslek verisini tutar.

Uygulama Adım 2

İkinci adım, okunan string değişkenleri ve sabit metinleri birleştirerek son tanıtım metnini oluşturmaktır.
Bu adımda string birleştirme (concatenation) tekniği kullanılır.



Kodlama Adım 2.1: Birleştirme Sentaksı

```
string tanitimMetni = 'Ad Soyad: ' + kullanıcıAdi + ' ' + kullanıcıSoyadi
```

Bu kısım, Ad Soyad etiketini ve kullanıcıdan alınan iki değişkeni birleştirir.

Kodlama Adım 2.2: Yeni Satır Karakteri

Metin içinde yeni bir satıra geçmek için özel bir kontrol karakteri olan kullanılır.

Bu karakter, formattaki okunabilirliği artırmak için kritiktir.

Kodlama Adım 2.3: Meslek Bilgisi Ekleme



Uygulama Adım 3

Son adım, oluşturulan nihai tanıtım metnini (tanitimMetni değişkeni) sonuç etiketine (lblKart) aktarmaktır.



Kodlama Adım 3.1: Sonuç Aktarımı

```
lblKart.Text = tanitimMetni;
```

Bu kod, çoklu satır formatındaki birleştirilmiş stringi ekranda görüntüler.

Önemli İpuçları ve Notlar

Profesyonel programlamada, sadece kodun çalışması değil, aynı zamanda okunabilir ve sürdürülebilir olması da önemlidir.
Bu durum, 'temiz kod' (clean code) prensiplerinin temelini oluşturur.



Değişken İsimlendirmenin Önemi I

Değişken isimlendirme, yazılım kalitesini doğrudan etkileyen kritik bir unsurdur.
İsimler mutlaka anlamlı olmalıdır.



Değişken İsimlendirmenin Önemi II

Anlamsız kısaltmalar yerine, değişkenin neyi temsil ettiğini açıkça belirten isimler kullanılmalıdır.
Örnek: 'x' veya 'y' yerine 'kullaniciAdi' veya 'fiyat' gibi isimler tercih edilmelidir.



Değişken İsimlendirme Kuralları I

Değişken isimleri içinde boşluk karakteri kesinlikle kullanılamaz. Programlama dili sentaksına özgü kısıtlamalar nedeniyle, Türkçe karakterlerin kullanımı da genellikle önerilmez ve kaçınılmalıdır.



Değişken İsimlendirme Kuralları II

Değişken isimleri mutlaka bir harf karakteriyle başlamalıdır.
Sayılar veya özel sembollerle başlamak programlama dilinde hata veya sentaks ihlaline neden olur.



Temiz Kodun İlk Adımı

Anlamlı deęişken isimlendirme kurallarına uymak, genel 'temiz kod' felsefesini benimsemenin ilk ve en temel adımıdır.
Bu, kodun başkaları tarafından kolayca anlaşılmasını sağlar.

Özet: Değişkenlerin Yapısal Rolü

Değişkenler, bir programın durumunu dinamik olarak koruyan ve veriye isim veren soyut yapılardır.
Bunlar, ya doğrudan veriyi içeren kutular ya da veriye işaret eden etiketler olarak işlev görür.



Özet: Veri Tipi Zorunluluğu

Her değişkenin bir veri tipi olmalıdır, bu da hangi tür bilginin saklanabileceğini ve hangi işlemlerin yapılabileceğini sınırlar. Bu, hataları azaltarak programın güvenilirliğini artırır.

Özet: string Veri Tipi

string veri tipi, karakter dizilerini (metinleri) depolamak için ayrılmıştır. Programlamada metinsel çıktıların üretilmesi ve kullanıcıdan metin girdilerinin alınması için temel araçtır.



Özet: Temel Operasyonlar

Temel işlemler şunlardır:

1. Tanımlama: Veri tipini ve ismi belirleme (string musterAdi;).
2. Atama: Değer yükleme (musterAdi = 'Veli Yılmaz';).
3. Birleştirme (Concatenation): (+) operatörü ile metinleri yan yana getirme.

İleri Kavram: Referans Tipleri Giriş

Unutulmamalıdır ki, değişkenler veriyi tutmaktan ziyade ona işaret edebilir. Bu ileri anlayış, string gibi daha karmaşık veri tiplerinin bellekte nasıl yönetildiğini anlamak için zemin hazırlar.

string Kullanımında Dikkat Edilmesi Gerekenler

string değerleri her zaman çift tırnak içinde (") yazılmalıdır.
Metin içinde özel karakterler kullanmak gerektiğinde gibi kaçış dizileri (escape sequences) kullanılır.

Haftalık Proje Değerlendirmesi

Kişisel Tanıtım Kartı Formu, string değişken tanımlama ve birleştirme işlemlerini somutlaştıran etkili bir projedir.
Öğrenciler, farklı değişken kombinasyonları ile metin üretmeyi pratik etmelidir.



C#'ta sentaks, derleyicinin kodu doğru yorumlaması için kritik öneme sahiptir.

`string kullanıcıAdı = txtAd.Text;` yapısı, dilin standart tip tanımını ve atama operatörünü birleştirir.



string Tipinin Sınırlamaları

string tipleri, doğrudan matematiksel işlem yapmak için uygun değildir. Eğer bir string sayı içeriyorsa bile, matematiksel işlemden önce uygun sayı tipine (int, double) dönüştürülmesi gerekir.

Görsel programlamada değişkenler, kullanıcı arayüzü bileşenlerinden (TextBox) verileri çekmek ve sonuçları (Label) göstermek için bir köprü görevi görür.

Bu, olay odaklı programlamanın temelini oluşturur.



Temiz Kod Standardı Tekrarı

İsimlendirmede anlamlılık, boşluksuz ve Türkçe karaktersiz kullanım, profesyonel yazılım geliştirme için zorunluluktur. Bu kural, büyük projelerde hata ayıklamayı ve ekip çalışmasını kolaylaştırır.

Sonuç ve Kapanış

Değişkenler ve string veri tipi, herhangi bir programlama dilinin en temel başlangıç noktalarıdır.

Bu kavramların sağlam anlaşılması, ilerideki karmaşık veri yapıları ve algoritmalar için temel oluşturur.



DEĞİŞKENLER

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

